The Relay: Resilient Research Paper Assistant

Author: Udbhaw Anand | **Repository:** https://github.com/Udbhaw08/the-relay | **Deliverable Run ID:** `live_groq_verified` (Live GroqCloud + Stand-in Stress Verification)

1 1. What I Built and Why

I built **The Relay**, an agentic research assistant in Python answering technical engineering queries over three arXiv papers: *Attention Is All You Need* (1706.03762v7), *Moshi: A Speech-Text Foundation Model* (2410.00037v1), and *Dynamic Chunking for H-Net* (2507.07955v2).

The system executes four decoupled stages:

1. **Query Parser & Intent Guard:** Extracts technical keywords, identifies scope, and detects unanswerable questions early.
2. **Hybrid Passage Retrieval:** Combines BM25Okapi lexical matching with dense vector search (`all-MiniLM-L6-v2`), fused via Reciprocal Rank Fusion ($\text{RRF}(d) = \sum \frac{1}{60+r(d)}$), followed by Cross-Encoder re-ranking.
3. **Synthesis & Citation Drafter:** Enforces strict bracketed anchors `[Paper, Page X, Section Y]`.
4. **Citation Verification Guard:** Verifies claims against chunk provenance, triggering a single query-expansion loop-back on ungrounded claims and refusing unanswerable queries.

The One Decision That Shaped It Most: Enforcing **zero silent fallback with adaptive retrieval degradation**. Upstream provider limits are never masked by silently routing queries to unannounced third-party models. Instead, when rate limits or queue pressures mount, the orchestrator adaptively throttles retrieval from Setting B/C down to compact Setting A (reducing prompt context tokens by ∼70%), preserving service uptime while ensuring complete telemetry provenance.

2 2. The Real Service and My Stand-In

The system targets the **GroqCloud API** (`api.groq.com`) using model `openai/gpt-oss-20b`. Core retrieval evaluations and steady traffic were validated live against GroqCloud with real token consumption and measured TTFT. Stress load scenarios (burst contention, overload queue saturation, per-minute halved limits, daily exhaustion) were verified using our calibrated Stand-in Simulator to protect account quotas from destructive exhaustion.

Header Semantics & Safety Caps: Groq officially documents `x-ratelimit-limit-requests` as Requests Per Day (RPD), NOT RPM. `x-ratelimit-limit-tokens` corresponds to TPM. Local safety caps default conservatively (30 RPM, 8K TPM, 1K RPD, 200K TPD) to protect shared organization quotas. Live HTTP requests confirmed active quota headers (`x-ratelimit-remaining-requests: 995`, `x-ratelimit-remaining-tokens: 7917`).

Stand-in Methodology: Latency is generated via a truncated Gaussian distribution ($\mu = 1200\,\text{ms}$, $\sigma = 300\,\text{ms}$, $\min = 150\,\text{ms}$) with a token generation rate of 15 ms/token ($\approx 66\,\text{tok/s}$). Contention is modeled via:

$$\text{TTFT}_{\text{eff}} = \text{TTFT}_{\text{base}} \times \left(1 + 0.08\, N_{\text{in-flight}}^{1.15}\right)$$

TPM and TPD limits are enforced via real tracked rolling-window token consumption rather than manual booleans.

Table 1: Table 1: Published Groq Free-plan limits (documentation & live-verified headers)

Limit	Published Value	Source / How Found
RPM	30 RPM (Free plan)	Official Groq Docs
TPM	8,000 TPM	Groq Docs / Live Header `x-ratelimit-limit-tokens` (8000)
RPD	1,000 RPD	Groq Docs / Live Header `x-ratelimit-limit-requests` (1000)
TPD	200,000 TPD	Official Groq Docs
In-Flight	Dynamic (1–5 conc.)	Groq Cloud Documentation
Hit Code	HTTP 429	Groq API Error Specification
Recovery	Dynamic	Headers (`retry-after`, reset)

3 3. Results

Benchmark requests were executed under run ID `live_groq_verified`, logging to `data/runs/live_groq_verified/results.csv`.

Metric Provenance: Live Groq evaluations report exact provider token counts extracted directly from HTTP completion usage metadata (`CompletionUsage.prompt_tokens` and `completion_tokens`). Live steady requests achieved 100.0% success with 554.8 actual provider tokens per answer. Setting B live evaluation on Groq achieved 40.0% factual accuracy (4/10 answerable questions) and 100.0% correct refusal (3/3 unanswerable questions), with 538.8 actual provider tokens per answer. Cost is labeled *"N/A (Free Tier / unbilled)"*. Stress load scenarios (burst, sustained overload, per-minute halved, daily exhaustion) were executed via the calibrated Stand-in Simulator to protect account quotas from destructive exhaustion.

Denominator & Isolation Notice: Pure retrieval benchmarks execute with adaptive degradation disabled (`allow_degradation=False`) so Setting A cannot contaminate Settings B or C. Infrastructure failures (2 TPM throttle timeouts under live traffic) are excluded from factual accuracy denominators and reported separately. Setting B evaluated live against GroqCloud achieved **40.0% factual accuracy** (4/10 answerable questions) and **100.0% correct refusal** (3/3 unanswerable questions), with zero hallucinations.

Chosen Default Setting: Setting B (500 tokens, 5 fetched, hybrid RRF re-ranking) is the default. It balances semantic context depth against prompt token cost. Under rate-limit warnings, the engine dynamically degrades to **Setting A** (250 tokens, 3 fetched, BM25-only), shedding ∼70% of context tokens.

An Unexpected Number: Live steady load achieved **100.0% success** on GroqCloud despite aggressive token arrival bursts, with the rate-limiter transparently orchestrating backoff recovery and adaptive degradation.

4 4. Where the Time Goes

Profiling live request `steady_groq_002` (Total: **8,834.9 ms**):

Table 2: Table 2a: Latency Across Load Scenarios | Table 2b: Resilience Outcomes (Run ID: `live_groq_verified`)

Scenario	Type	Reqs	Duration (s)	Offered Rate	Simulated / Live TTFT p50 / p95 / p99 (ms)	Total Latency p50 / p95 / p99 (ms)
steady	groq (live)	5	133.7s (73.47s disp)	3.3 req/min (int 18.37s)	1,550.4 ms (live stream)	60,128.2 / 70,380.4 / 72,415.4
burst	stand-in	20	66.9s (59.55s disp)	Burst (20 concurrent)	2,295.9 / 3,317.7 / 3,699.5	6,068.6 / 57,986.6 / 58,025.1
sustained_overload	stand-in	25	118.6s (113.64s disp)	13.2 req/min	1,792.9 / 2,364.5 / 2,425.0	4,957.3 / 35,205.0 / 50,656.7
per_minute_halved	stand-in	15	188.7s (187.01s disp)	4.8 req/min	1,386.8 / 1,569.8 / 1,644.5	3,885.4 / 4,853.8 / 5,086.0
daily_exhausted	stand-in	10	2.0s	307.7 req/min	N/A	N/A

Scenario	Succeeded %	Rate-Limited %	Timed Out %	Retries/Req	Peak Queue	Tokens/Ans	Cost / 1,000 Answers
steady	100.0%	0.0%	0.0%	0.60	2	554.8 (actual)	N/A (Free Tier / unbilled)
burst	60.0%	35.0%	5.0%	0.90	20	35.3	N/A — no live provider usage
sustained_overload	36.0%	44.0%	20.0%	1.04	15	35.2	N/A — no live provider usage
per_minute_halved	80.0%	6.7%	13.3%	0.60	0	35.2	N/A — no live provider usage
daily_exhausted	0.0%	100.0%	0.0%	0.00	0	0.0	N/A — no live provider usage

Table 3: Table 3: Retrieval Settings Evaluation and Factual Accuracy Breakdown

Setting (Chunk, Fetch, Rerank)	Questions	Correct & Cited %	Wrong %	Correctly Refused %	Tokens/Ans	Simulated / Live p50/p95 (m
Setting A: 250 tok, 3 fetch, no rerank	15	8.3% (1/12)	91.7% (11/12)	100.0% (3/3)	39.1	1,418.0 / 1,55
Setting B: 500 tok, 5 fetch, rerank: yes **(Live Groq)**	15	**40.0% (4/10)**	60.0% (6/10)	**100.0% (3/3)**	**538.8 (actual)**	1,550.4 ms (li
Setting C: 1000 tok, 7 fetch, rerank: yes	15	8.3% (1/12)	91.7% (11/12)	100.0% (3/3)	33.1	1,418.0 / 1,57

Setting	Ans Eval	Correct & Cited	Partial	Wrong	Wrong Refused	Unans Eval	Correct Refused	Hallucinated	In
setting_a	12/12	1	0	11	0	3/3	3	0	
setting_b **(Live Groq)**	**10/12**	**4**	**0**	**6**	**0**	**3/3**	**3**	**0**	
setting_c	12/12	1	0	8	3	3/3	3	0	

- **Queue Wait Time:** 1.1 ms (admitted immediately).
- **Retrieval Time:** 7,538.9 ms (**85.3% of duration**). Dense embedding cold-start across all corpus chunk partitions.
- **Inference Time:** 1,295.2 ms (**14.7% of duration**: live Groq generation across 2,911 prompt tokens and 690 completion tokens).
- **Verification Time:** 0.8 ms (< 0.01%: regex extraction and page/anchor verification).
- **Warm Comparison:** Once vector weights are memory-resident (`steady_groq_004`), retrieval drops to **16.1 ms** (99.8% reduction), reducing total request latency to **1,314.8 ms**.

5 5. When a Limit Is Hit

Per-Minute Limit (RPM/TPM): Handled via an in-memory queue. Requests execute truncated exponential backoff with full jitter ($T \sim \mathcal{U}(0, \min(M, B \times 2^k))$) and adaptively throttle retrieval to Setting A (250 tokens, BM25-only), shedding ∼70% of prompt token load.

Per-Day Limit (RPD/TPD): Stops retries immediately (0 retries). Requests fail fast (< 1ms) with an explicit notice stating dynamic reset estimation, preventing thundering-herd queue lockup.

6 6. How This Library Will Mislead a Naive Retriever

1. **Lexical Collision on "Attention":** Queries regarding *Moshi's* temporal stream masking match Vaswani's translation masking in *Attention Is All You Need*, returning translation equations instead of audio synchronization.
2. **Term Collision on "Chunking":** Queries on dynamic chunking benefits over tokenizers match *Moshi's* 80ms acoustic frame stride rather than *H-Net's* byte-level hierarchy.
3. **Vocabulary Mismatch on "Tokens":** Asking about discrete audio speech tokens retrieves BPE subwords from Vaswani et al. rather than Mimi neural codec RVQ codebooks.

7 7. Retries

Policy: Truncated exponential backoff with true full jitter ($T \sim \mathcal{U}(0, \min(30.0, 2.0 \times 2^k))$), maximum 2 retries, respecting provider `Retry-After` headers up to a 60s safety cap.

Where Retrying Made Things Worse: Retrying on **daily budget exhaustion** caused catastrophic queue saturation. Classifying daily exhaustion as a non-retryable error eliminated retry loops completely.

8 8. What the Person Sees

When daily quota is depleted, the user receives an explicit, transparent notice:

> *"[Notice] Daily Query Budget Exhausted: The system's daily LLM inference capacity has reached its limit for today. The next quota allocation resets dynamically (or when upstream headers indicate). Your request was rejected locally to protect shared quotas. Please consult the local paper index or retry once the quota window refreshes."*

Note on Extended Analysis: Comprehensive phase execution matrices, architectural diagrams, extended reflections, feedback on Vaani, and assistant collaboration logs are documented in `docs/implementation/FINAL_REPORT.md`.